TASK 1

This one actually works: import os

```
import numpy as np
import autogalaxy as ag
import autogalaxy.plot as aplt

# Define dataset path
dataset_path_1 = os.path.join("dataset", "task_3b_mergers", "merger_1")
dataset_path_2 = os.path.join("dataset", "task_3b_mergers", "merger_2")

# Load imaging data
def load_dataset(dataset_path, pixel_scales=0.06):
    if not os.path.exists(dataset_path):
        print(f"Warning: Dataset path {dataset_path} not found.")
        return None
    try:
        return ag.Imaging.from_fits(
            data_path=os.path.join(dataset_path, "data.fits"),
            noise_map_path=os.path.join(dataset_path, "noise_map.fits"),
            psf_path=os.path.join(dataset_path, "psf.fits"),
            pixel_scales=pixel_scales,
        )
    except FileNotFoundError as e:
        print(f"Error: {e}")
        return None

dataset_1 = load_dataset(dataset_path_1)
dataset_2 = load_dataset(dataset_path_2)

if dataset_1 is not None:
    dataset_plotter_1 = aplt.ImagingPlotter(dataset=dataset_1)
    dataset_plotter_1.subplot_dataset()

if dataset_2 is not None:
    dataset_plotter_2 = aplt.ImagingPlotter(dataset=dataset_2)
    dataset_plotter_2.subplot_dataset()

# Define and fit Sersic profiles
```

```
def fit_sersic_light_profile(dataset):
    if dataset is None:
        return None, None

    grid = ag.Grid2D.uniform(
        shape_native=dataset.shape_native,
pixel_scales=dataset.pixel_scales
    )

    sersic_light_profile_1 = ag.lp.Sersic(
        centre=(-0.5, 0.0), ell_comps=(0.0, 0.1), intensity=1.0,
effective_radius=2.0, sersic_index=1.0
    )

    sersic_light_profile_2 = ag.lp.Sersic(
        centre=(0.5, 0.0), ell_comps=(0.0, 0.2), intensity=0.5,
effective_radius=1.5, sersic_index=1.5
    )

    image_1 = sersic_light_profile_1.image_2d_from(grid=grid)
    image_2 = sersic_light_profile_2.image_2d_from(grid=grid)

    total_intensity_1 = np.sum(image_1)
    total_intensity_2 = np.sum(image_2)

    intensity_ratio = total_intensity_1 / total_intensity_2

    return intensity_ratio, image_1 + image_2

# Fit profiles and verify intensity ratios
if dataset_1 is not None:
    intensity_ratio_1, model_image_1 = fit_sersic_light_profile(dataset_1)
    print(f"Intensity ratio for merger_1: {intensity_ratio_1} (expected
~2.0)")
    if model_image_1 is not None:
        array_plotter_1 = aplt.Array2DPlotter(array=model_image_1)
        array_plotter_1.set_title("Fitted Model - Merger 1")
        array_plotter_1.figure_2d()

if dataset_2 is not None:
```

```
    intensity_ratio_2, model_image_2 = fit_sersic_light_profile(dataset_2)
    print(f"Intensity ratio for merger_2: {intensity_ratio_2} (expected
~5.0)")
    if model_image_2 is not None:
        array_plotter_2 = aplt.Array2DPlotter(array=model_image_2)
        array_plotter_2.set_title("Fitted Model - Merger 2")
        array_plotter_2.figure_2d()
```

Apparently this one aliens more with the tutorial code: `import os`

```
import numpy as np
import autogalaxy as ag
import autogalaxy.plot as aplt

# Define dataset path
dataset_path = os.path.join("dataset", "task_3b_mergers", "merger_1")

# Load imaging data
dataset = ag.Imaging.from_fits(
    data_path=os.path.join(dataset_path, "data.fits"),
    noise_map_path=os.path.join(dataset_path, "noise_map.fits"),
    psf_path=os.path.join(dataset_path, "psf.fits"),
    pixel_scales=0.06,
)

# Plot dataset image
dataset_plotter = aplt.ImagingPlotter(dataset=dataset)
dataset_plotter.subplot_dataset()

# Define and fit Sersic profile
grid = ag.Grid2D.uniform(
    shape_native=dataset.shape_native, pixel_scales=dataset.pixel_scales
)
```

```
sersic_light_profile = ag.lp.Sersic(
    centre=(0.0, 0.0), ell_comps=(0.0, 0.1), intensity=1.0,
effective_radius=2.0, sersic_index=1.0
)

image = sersic_light_profile.image_2d_from(grid=grid)

total_intensity = np.sum(image)
print(f"Total intensity: {total_intensity}")

# Plot fitted model
array_plotter = aplt.Array2DPlotter(array=image)
array_plotter.set_title("Fitted Sersic Model - Merger 1")
array_plotter.figure_2d()
```

TASK 2

```
import numpy as np
from os import path
import autogalaxy as ag
import autogalaxy.plot as aplt

dataset_path = path.join("dataset", "task_3b_mergers", "jwst",
"dataset_0")

def load_dataset(dataset_path):
    return ag.Imaging.from_fits(
        data_path=os.path.join(dataset_path, "data.fits"),
        noise_map_path=os.path.join(dataset_path, "noise_map.fits"),
        psf_path=os.path.join(dataset_path, "psf.fits"),
        pixel_scales=0.06,
    )
```

```
dataset = load_dataset(dataset_path)

# Plot dataset images
dataset_plotter_1 = aplt.ImagingPlotter(dataset=dataset)
dataset_plotter_1.subplot_dataset()
```

```
import os
import numpy as np
import autogalaxy as ag
import autogalaxy.plot as aplt
import autofit as af

# Define JWST dataset path
jwst_dataset_path = os.path.join("dataset", "task_3b_mergers", "jwst")

# List all dataset folders within JWST folder
jwst_folders = [f for f in os.listdir(jwst_dataset_path) if
os.path.isdir(os.path.join(jwst_dataset_path, f))]

# Select three merging galaxies for analysis
selected_folders = [jwst_folders[9], jwst_folders[1], jwst_folders[4]]  #
Ensure correct indexing

# Function to load dataset from each folder
def load_dataset(dataset_path):
    return ag.Imaging.from_fits(
        data_path=os.path.join(dataset_path, "data.fits"),
        noise_map_path=os.path.join(dataset_path, "noise_map.fits"),
        psf_path=os.path.join(dataset_path, "psf.fits"),
        pixel_scales=0.06,
    )

# Function to apply a circular mask
```

```
def apply_mask(dataset):
    mask = ag.Mask2D.circular(
        shape_native=dataset.shape_native,
        pixel_scales=dataset.pixel_scales,
        radius=1.5,
        centre=(0.0, 0.0),
    )
    return dataset.apply_mask(mask=mask)

# Function to create a model with priors
def fit_sersic_light_profiles():
    galaxy_model = af.Model(ag.Galaxy, redshift=0.5,
bulge=ag.lp_linear.Sersic)
    model = af.Collection(galaxies=af.Collection(galaxy=galaxy_model))

    # Set priors based on tutorial examples
    model.galaxies.galaxy.bulge.centre.centre_0 =
af.UniformPrior(lower_limit=-0.1, upper_limit=0.1)
    model.galaxies.galaxy.bulge.centre.centre_1 =
af.UniformPrior(lower_limit=-0.1, upper_limit=0.1)
    model.galaxies.galaxy.bulge.ell_comps.ell_comps_0 =
af.UniformPrior(lower_limit=-0.5, upper_limit=0.5)
    model.galaxies.galaxy.bulge.ell_comps.ell_comps_1 =
af.UniformPrior(lower_limit=-0.5, upper_limit=0.5)
    model.galaxies.galaxy.bulge.effective_radius =
af.UniformPrior(lower_limit=0.0, upper_limit=10.0)
    model.galaxies.galaxy.bulge.sersic_index =
af.UniformPrior(lower_limit=0.8, upper_limit=8.0)

    return model

# Function to run analysis, compute intensity ratio, and plot residuals
def analyze_galaxy(dataset, model):
    search = af.Nautilus(
        name="sersic_fit",
        n_live=30,
        max_iterations=100,
        iterations_per_update=50,
        force_x1_cpu=True  # Ensures correct execution in environments
like Google Colab
```

```
    )

    analysis = ag.AnalysisImaging(dataset=dataset)
    result = search.fit(model=model, analysis=analysis)

    fit = result.max_log_likelihood_fit

    print(f"Best-fit parameters for dataset:")
    print(fit.instance.galaxies.galaxy.bulge)

    # Compute intensity ratio
    total_intensity_1 = np.sum(fit.model_image.native)
    total_intensity_2 = np.sum(fit.data.native)
    intensity_ratio = total_intensity_1 / total_intensity_2

    print(f"Intensity Ratio: {intensity_ratio:.2f}")

    # Plot best-fit Sersic profile
    grid = ag.Grid2D.uniform(shape_native=(50, 50),
pixel_scales=dataset.pixel_scales)
    light_profile_plotter =
aplt.LightProfilePlotter(light_profile=fit.instance.galaxies.galaxy.bulge,
grid=grid)
    light_profile_plotter.set_title("Fitted Sersic Light Profile")
    light_profile_plotter.figures_2d(image=True)
    light_profile_plotter.figures_1d(image=True)

    fit_plotter = aplt.FitImagingPlotter(fit=fit)
    fit_plotter.subplot_fit()  # Display data, model, residuals, and
chi-squared map

    return intensity_ratio

# Analyze selected galaxies and record intensity ratios
intensity_ratios = []
for folder in selected_folders:
    dataset_path = os.path.join(jwst_dataset_path, folder)
    dataset = load_dataset(dataset_path)
    dataset = apply_mask(dataset)
    model = fit_sersic_light_profiles()
```

```
    print(f"Galaxy {folder} - Running Fit")
    intensity_ratio = analyze_galaxy(dataset, model)
    intensity_ratios.append((folder, intensity_ratio))

# Print final intensity ratio range
ratios = [r[1] for r in intensity_ratios]
print(f"Final Intensity Ratio Range: {min(ratios):.2f} -
{max(ratios):.2f}")
```

Task 3

```
import os
import numpy as np
import autogalaxy as ag
import autogalaxy.plot as aplt
import autofit as af
import matplotlib.pyplot as plt

# Define paths to find the datasets
def locate_datasets():
    """Find directories containing galaxy merger datasets"""
    # Possible base paths for datasets
    base_paths = [
        "dataset",
        "/content/BSc_Galaxies_Project/dataset",
    ]

    # Possible subdirectories for JWST data
    jwst_subdirs = [
        "task_3b_mergers/jwst",
        "task_3a_mergers",
        "imaging/mergers"
    ]
```

```
    # Check all possible combinations
    for base in base_paths:
        for subdir in jwst_subdirs:
            full_path = os.path.join(base, subdir)
            if os.path.exists(full_path):
                print(f"Found dataset directory: {full_path}")
                return full_path

    # If no valid path found, try the current directory structure
    print("Could not find expected dataset directories. Looking for any
available datasets...")
    for base in base_paths:
        if os.path.exists(base):
            print(f"Found base directory: {base}")
            # List contents of base directory
            contents = os.listdir(base)
            if contents:
                print(f"Contents of {base}: {contents}")
            return base

    raise FileNotFoundError("Could not locate any dataset directories")

# Find available datasets
dataset_base_path = locate_datasets()

def list_available_galaxies(base_path):
    """List all available galaxy datasets"""
    available_galaxies = []

    # Try to explore the directory structure
    try:
        # First, check if we're already in a JWST directory
        if os.path.exists(os.path.join(base_path, "data.fits")):
            # This is already a dataset directory
            return [os.path.basename(base_path)]

        # Check direct subdirectories
        for item in os.listdir(base_path):
            item_path = os.path.join(base_path, item)
            if os.path.isdir(item_path):
```

```
                # Check if this directory has FITS files
                if os.path.exists(os.path.join(item_path, "data.fits")):
                    available_galaxies.append(item)
                else:
                    # Check one level deeper
                    for subitem in os.listdir(item_path):
                        subitem_path = os.path.join(item_path, subitem)
                        if os.path.isdir(subitem_path) and
os.path.exists(os.path.join(subitem_path, "data.fits")):
                            available_galaxies.append(os.path.join(item,
subitem))
    except Exception as e:
        print(f"Error exploring datasets: {e}")

    return available_galaxies

# List available galaxies
available_galaxies = list_available_galaxies(dataset_base_path)
print(f"Found {len(available_galaxies)} galaxy datasets:
{available_galaxies}")

def load_dataset(galaxy_name, base_path):
    """Load a galaxy dataset by name"""
    # Construct full path to the dataset
    dataset_path = os.path.join(base_path, galaxy_name)

    # If the name includes subdirectories, make sure the path is correct
    if not os.path.exists(dataset_path):
        # Try finding it in the directory structure
        for root, dirs, files in os.walk(base_path):
            if os.path.basename(root) == galaxy_name:
                dataset_path = root
                break

    print(f"Loading dataset from: {dataset_path}")

    # Verify files exist
    data_path = os.path.join(dataset_path, "data.fits")
    noise_map_path = os.path.join(dataset_path, "noise_map.fits")
    psf_path = os.path.join(dataset_path, "psf.fits")
```

```
    if not os.path.exists(data_path):
        raise FileNotFoundError(f"Could not find data.fits in
{dataset_path}")

    # Check if noise and PSF files exist
    has_noise = os.path.exists(noise_map_path)
    has_psf = os.path.exists(psf_path)

    print(f"Found files: data={os.path.exists(data_path)},
noise={has_noise}, psf={has_psf}")

    # Load the dataset with appropriate parameters
    pixel_scales = 0.06  # Default for JWST data, adjust if needed

    try:
        if has_noise and has_psf:
            dataset = ag.Imaging.from_fits(
                data_path=data_path,
                noise_map_path=noise_map_path,
                psf_path=psf_path,
                pixel_scales=pixel_scales,
            )
        elif has_psf:
            dataset = ag.Imaging.from_fits(
                data_path=data_path,
                psf_path=psf_path,
                pixel_scales=pixel_scales,
            )
        else:
            dataset = ag.Imaging.from_fits(
                data_path=data_path,
                pixel_scales=pixel_scales,
            )

        return dataset
    except Exception as e:
        print(f"Error loading dataset: {e}")
        raise
```

```
def create_mask(dataset, radius=1.5, centre=(0.0, 0.0)):
    """Create a circular mask around the galaxy"""
    mask = ag.Mask2D.circular(
        shape_native=dataset.shape_native,
        pixel_scales=dataset.pixel_scales,
        radius=radius,
        centre=centre
    )

    return dataset.apply_mask(mask=mask)

def visualize_data(dataset, title="Galaxy Merger Data"):
    """Visualize the dataset"""
    dataset_plotter = aplt.ImagingPlotter(dataset=dataset)
    dataset_plotter.set_title(title)
    dataset_plotter.subplot_dataset()

    return dataset_plotter

def setup_single_galaxy_model():
    """Setup a single galaxy model with linear light profile"""
    galaxy_model = af.Model(ag.Galaxy, redshift=0.5,
bulge=ag.lp_linear.Sersic)
    model = af.Collection(galaxies=af.Collection(galaxy=galaxy_model))

    # Set tight priors for faster convergence
    model.galaxies.galaxy.bulge.centre.centre_0 = af.UniformPrior(-0.2,
0.2)
    model.galaxies.galaxy.bulge.centre.centre_1 = af.UniformPrior(-0.2,
0.2)
    model.galaxies.galaxy.bulge.ell_comps.ell_comps_0 =
af.UniformPrior(-0.3, 0.3)
    model.galaxies.galaxy.bulge.ell_comps.ell_comps_1 =
af.UniformPrior(-0.3, 0.3)
    model.galaxies.galaxy.bulge.effective_radius = af.UniformPrior(0.1,
2.0)
    model.galaxies.galaxy.bulge.sersic_index = af.UniformPrior(0.8, 4.0)

    return model
```

```
def setup_merger_model():
    """Setup a two-component model for merging galaxies"""
    # First galaxy (primary)
    galaxy_1 = af.Model(ag.Galaxy, redshift=0.5,
bulge=ag.lp_linear.Sersic)

    # Second galaxy (merging)
    galaxy_2 = af.Model(ag.Galaxy, redshift=0.5,
bulge=ag.lp_linear.Sersic)

    # Collection of both galaxies
    model = af.Collection(
        galaxies=af.Collection(
            galaxy_1=galaxy_1,
            galaxy_2=galaxy_2
        )
    )

    # Set tight priors for faster convergence
    # Galaxy 1
    model.galaxies.galaxy_1.bulge.centre.centre_0 = af.UniformPrior(-0.5,
0.5)
    model.galaxies.galaxy_1.bulge.centre.centre_1 = af.UniformPrior(-0.5,
0.5)
    model.galaxies.galaxy_1.bulge.ell_comps.ell_comps_0 =
af.UniformPrior(-0.3, 0.3)
    model.galaxies.galaxy_1.bulge.ell_comps.ell_comps_1 =
af.UniformPrior(-0.3, 0.3)
    model.galaxies.galaxy_1.bulge.effective_radius = af.UniformPrior(0.1,
2.0)
    model.galaxies.galaxy_1.bulge.sersic_index = af.UniformPrior(0.8, 4.0)

    # Galaxy 2
    model.galaxies.galaxy_2.bulge.centre.centre_0 = af.UniformPrior(-0.5,
0.5)
    model.galaxies.galaxy_2.bulge.centre.centre_1 = af.UniformPrior(-0.5,
0.5)
    model.galaxies.galaxy_2.bulge.ell_comps.ell_comps_0 =
af.UniformPrior(-0.3, 0.3)
```

```
    model.galaxies.galaxy_2.bulge.ell_comps.ell_comps_1 =
af.UniformPrior(-0.3, 0.3)
    model.galaxies.galaxy_2.bulge.effective_radius = af.UniformPrior(0.1,
2.0)
    model.galaxies.galaxy_2.bulge.sersic_index = af.UniformPrior(0.8, 4.0)

    return model

def run_fast_fit(dataset, model, name="merger_fit", use_multinest=False):
    """Run a fast fit with optimized settings"""
    if use_multinest:
        # Use MultiNest for potentially faster convergence
        search = af.Nautilus(
            name=name,
            n_live=20,  # Fewer live points for speed
            max_iterations=50,  # Limited iterations
            iterations_per_update=25,
            force_x1_cpu=True,
        )
    else:
        # Use Dynesty with optimized settings for speed
        search = af.DynestyStatic(
            name=name,
            nlive=20,  # Reduced for speed
            walks=5,
            dlogz=1.0,
            maxcall=500,  # Limited iterations
            force_x1_cpu=True,
        )

    # Setup analysis
    analysis = ag.AnalysisImaging(dataset=dataset)

    # Run the fit
    print("Running fit - this may take a minute...")
    result = search.fit(model=model, analysis=analysis)

    return result

def calculate_intensity_ratio(fit):
```

```
    """Calculate intensity ratio between merger components"""
    if hasattr(fit.instance.galaxies, 'galaxy_1') and
hasattr(fit.instance.galaxies, 'galaxy_2'):
        # For two-component fit
        # Get intensity from linear light profiles directly
        galaxy_1 = fit.instance.galaxies.galaxy_1
        galaxy_2 = fit.instance.galaxies.galaxy_2

        # Get intensities from the linear light profile dict
        intensity_1 =
fit.linear_light_profile_intensity_dict[galaxy_1.bulge]
        intensity_2 =
fit.linear_light_profile_intensity_dict[galaxy_2.bulge]

        # Calculate flux (intensity × effective_radius²)
        flux_1 = intensity_1 * galaxy_1.bulge.effective_radius**2
        flux_2 = intensity_2 * galaxy_2.bulge.effective_radius**2

        return {
            "intensity_ratio": intensity_1 / intensity_2,
            "flux_ratio": flux_1 / flux_2
        }
    else:
        # For single-component fit, compare model to data
        model_sum = np.sum(fit.model_data)
        data_sum = np.sum(fit.data)

        return {
            "model_to_data_ratio": model_sum / data_sum
        }

def visualize_sersic_profile(light_profile, grid, title="Sersic Light
Profile"):
    """Visualize a Sersic light profile"""
    light_profile_plotter = aplt.LightProfilePlotter(
        light_profile=light_profile,
        grid=grid
    )

    # 2D visualization
```

```
    light_profile_plotter.set_title(f"{title} - 2D")
    light_profile_plotter.figures_2d(image=True)

    # 1D radial profile
    light_profile_plotter.set_title(f"{title} - 1D Radial Profile")
    light_profile_plotter.figures_1d(image=True)

    # Log-scaled version for better visualization
    light_profile_plotter = aplt.LightProfilePlotter(
        light_profile=light_profile,
        grid=grid,
        mat_plot_2d=aplt.MatPlot2D(use_log10=True),
    )
    light_profile_plotter.set_title(f"{title} - Log Scale")
    light_profile_plotter.figures_2d(image=True)

def visualize_results(fit, title="Galaxy Fit Results"):
    """Visualize the fit results"""
    fit_plotter = aplt.FitImagingPlotter(fit=fit)
    fit_plotter.set_title(title)
    fit_plotter.subplot_fit()

    # Create a uniform grid for light profile visualization
    grid = ag.Grid2D.uniform(
        shape_native=(101, 101),
        pixel_scales=fit.dataset.pixel_scales,
    )

    # Check what type of fit we have (single or double component)
    if hasattr(fit.instance.galaxies, 'galaxy_1') and
hasattr(fit.instance.galaxies, 'galaxy_2'):
        # For two-component fit, visualize each component
        galaxy_1 = fit.instance.galaxies.galaxy_1
        galaxy_2 = fit.instance.galaxies.galaxy_2

        # Visualize individual components
        visualize_sersic_profile(galaxy_1.bulge, grid, title=f"{title} -
Galaxy 1")
        visualize_sersic_profile(galaxy_2.bulge, grid, title=f"{title} -
Galaxy 2")
```

```
        # Plot galaxies together
        galaxies_plotter =
aplt.GalaxiesPlotter(galaxies=fit.instance.galaxies, grid=grid)
        galaxies_plotter.set_title(f"{title} - Combined Galaxies")
        galaxies_plotter.subplot_galaxy_images()
    else:
        # For single-component fit
        galaxy = fit.instance.galaxies.galaxy
        visualize_sersic_profile(galaxy.bulge, grid, title=f"{title} -
Galaxy")

    return fit_plotter

def analyze_merger(galaxy_name, base_path, use_two_component=True):
    """Full analysis workflow for a single merger"""
    print(f"\n=== Analyzing Galaxy Merger: {galaxy_name} ===")

    try:
        # Load data
        dataset = load_dataset(galaxy_name, base_path)

        # Visualize raw data
        visualize_data(dataset, title=f"Galaxy {galaxy_name} Raw Data")

        # Apply mask
        masked_dataset = create_mask(dataset, radius=1.5)
        visualize_data(masked_dataset, title=f"Galaxy {galaxy_name} Masked
Data")

        # Setup model based on chosen approach
        if use_two_component:
            model = setup_merger_model()
            model_type = "two-component"
        else:
            model = setup_single_galaxy_model()
            model_type = "single-component"

        # Run fit
        result = run_fast_fit(
```

```
            masked_dataset,
            model,
            name=f"{galaxy_name}_{model_type}",
            use_multinest=True  # Try Nautilus/MultiNest which can be
faster
        )

        # Get best fit
        fit = result.max_log_likelihood_fit

        # Calculate and print intensity ratio
        ratios = calculate_intensity_ratio(fit)
        print(f"\nResults for Galaxy {galaxy_name} ({model_type}):")
        for ratio_name, ratio_value in ratios.items():
            print(f"{ratio_name} = {ratio_value:.2f}")

        # Print best-fit parameters
        if use_two_component:
            print(f"\nGalaxy 1 (Primary) Parameters:")
            galaxy_1 = fit.instance.galaxies.galaxy_1
            print(f"Centre: {galaxy_1.bulge.centre}")
            print(f"Effective Radius: {galaxy_1.bulge.effective_radius}")
            print(f"Sersic Index: {galaxy_1.bulge.sersic_index}")
            print(f"Intensity:
{fit.linear_light_profile_intensity_dict[galaxy_1.bulge]}")

            print(f"\nGalaxy 2 (Secondary) Parameters:")
            galaxy_2 = fit.instance.galaxies.galaxy_2
            print(f"Centre: {galaxy_2.bulge.centre}")
            print(f"Effective Radius: {galaxy_2.bulge.effective_radius}")
            print(f"Sersic Index: {galaxy_2.bulge.sersic_index}")
            print(f"Intensity:
{fit.linear_light_profile_intensity_dict[galaxy_2.bulge]}")
        else:
            print(f"\nGalaxy Parameters:")
            galaxy = fit.instance.galaxies.galaxy
            print(f"Centre: {galaxy.bulge.centre}")
            print(f"Effective Radius: {galaxy.bulge.effective_radius}")
            print(f"Sersic Index: {galaxy.bulge.sersic_index}")
```

```
            print(f"Intensity:
{fit.linear_light_profile_intensity_dict[galaxy.bulge]}")

        # Visualize results
        visualize_results(fit, title=f"Galaxy {galaxy_name} Fit
({model_type})")

        return {
            "galaxy_name": galaxy_name,
            "model_type": model_type,
            "ratios": ratios,
            "fit": fit
        }
    except Exception as e:
        print(f"Error analyzing galaxy {galaxy_name}: {e}")
        import traceback
        traceback.print_exc()
        return None

# Main execution block

# If we found galaxy datasets, analyze a few of them
if available_galaxies:
    # Select up to 3 galaxies to analyze (or use all if less than 3)
    selected_galaxies = available_galaxies[:min(3,
len(available_galaxies))]

    print(f"\nSelected galaxies for analysis: {selected_galaxies}")

    # Try both single and two-component fits
    results = []

    for galaxy_name in selected_galaxies:
        # First try single component (faster)
        single_result = analyze_merger(galaxy_name, dataset_base_path,
use_two_component=False)
        if single_result:
            results.append(single_result)

        # Then try two-component if you want and have time
```

```
        # two_comp_result = analyze_merger(galaxy_name, dataset_base_path,
use_two_component=True)
        # if two_comp_result:
        #     results.append(two_comp_result)

    # Summarize findings
    if results:
        print("\n=== Summary of Merger Analysis ===")
        print("Galaxy | Model Type | Intensity/Flux Ratio")
        print("-" * 50)

        for r in results:
            ratios_str = " | ".join([f"{k}={v:.2f}" for k, v in
r['ratios'].items()])
            print(f"{r['galaxy_name']} | {r['model_type']} |
{ratios_str}")

        # Calculate range of intensity ratios if we have comparable values
        if all('intensity_ratio' in r['ratios'] for r in results):
            intensity_ratios = [r['ratios']['intensity_ratio'] for r in
results if 'intensity_ratio' in r['ratios']]
            print(f"\nRange of intensity ratios:
{min(intensity_ratios):.2f} - {max(intensity_ratios):.2f}")
    else:
        print("No successful analyses completed.")
else:
    print("No galaxy datasets found. Please check your directory
structure.")
```

```python
import os
import numpy as np
import autogalaxy as ag
import autogalaxy.plot as aplt
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter
from skimage.feature import peak_local_max

# Define JWST dataset base path
jwst_dataset_path = os.path.join("dataset", "task_1b_mergers", "jwst")

# Create output directories
os.makedirs("output/plots", exist_ok=True)
os.makedirs("output/results", exist_ok=True)

# Get all available datasets
available_datasets = sorted([f for f in os.listdir(jwst_dataset_path)
                             if os.path.isdir(os.path.join(jwst_dataset_path, f))])
print(f"Found {len(available_datasets)} datasets: {available_datasets}")

def load_dataset(folder_name):
    """Load imaging data from FITS files"""
    dataset_path = os.path.join(jwst_dataset_path, folder_name)
    return ag.Imaging.from_fits(
        data_path=os.path.join(dataset_path, "data.fits"),
        noise_map_path=os.path.join(dataset_path, "noise_map.fits"),
        psf_path=os.path.join(dataset_path, "psf.fits"),
        pixel_scales=0.06,
    )

def find_galaxy_centers(dataset, min_distance=15, num_peaks=2, sigma=2.0):
    """Improved galaxy center detection using peak finding"""
    data = dataset.data.native
    smoothed = gaussian_filter(data, sigma=sigma)

    # Find peaks in the smoothed data
    peaks = peak_local_max(
        smoothed,
        min_distance=min_distance,
        num_peaks=num_peaks,
        threshold_rel=0.2,
        exclude_border=int(min_distance/2)
    )

    # Convert to physical coordinates (arcsec)
    center_y, center_x = np.array(data.shape) / 2
    pixel_scale = dataset.pixel_scales[0]
    centers = []

    for y, x in peaks:
        rel_x = (x - center_x) * pixel_scale
        rel_y = (center_y - y) * pixel_scale
        centers.append((rel_x, rel_y))

    # If we didn't find enough peaks, add reasonable offsets
    while len(centers) < 2:
        if not centers:
            # If no peaks found at all, use image center
            centers.append((0.0, 0.0))
        # Add offset from last found center
        last_x, last_y = centers[-1]
        centers.append((last_x + 0.3, last_y + 0.3)  # 0.5 arcsec offset

    return centers[:2]  # Return exactly two centers

def plot_centers(data, centers, filename):
    """Visual verification of detected centers"""
    plt.figure(figsize=(8, 8))
    plt.imshow(np.log10(data + 1e-10), cmap='viridis')

    # Convert centers back to pixel coordinates for plotting
    center_y, center_x = np.array(data.shape) / 2
    pixel_scale = 0.06
    for i, (x, y) in enumerate(centers):
        px = x/pixel_scale + center_x
        py = -y/pixel_scale + center_y
        plt.scatter(px, py, s=300, marker='x', colors='red', linewidths=3)
        plt.text(px, py, f" G{i+1}", colors='white', fontsize=12)

    plt.colorbar(label='log10(Flux)')
    plt.title("Detected Galaxy Centers")
    plt.savefig(filename, bbox_inches='tight')
    plt.show()
    plt.close()

def analyze_single_merger(dataset_name,
                          # Galaxy 1 parameters
                          g1_ell_comps=(0.1, 0.1),
                          g1_intensity=1.0,
                          g1_effective_radius=0.4,
                          g1_sersic_index=1.0,
                          # Galaxy 2 parameters
                          g2_ell_comps=(0.1, 0.0),
                          g2_intensity=2.0,
                          g2_effective_radius=0.4,
                          g2_sersic_index=2.0,
                          # Peak finding parameters
                          min_distance=15,
                          sigma=2.0,
                          # Mask parameters
                          mask_radius=3.0):

    print(f"\n{'='*70}")
    print(f"Analyzing {dataset_name}")

    try:
        # Load the dataset
        dataset = load_dataset(dataset_name)

        # 1. Find galaxy centers with customizable parameters
        centers = find_galaxy_centers(
            dataset,
            min_distance=min_distance,
            num_peaks=2,
            sigma=sigma
        )

        separation = np.sqrt((centers[0][0]-centers[1][0])**2 +
                             (centers[0][1]-centers[1][1])**2)
        print(f"Detected centers: {centers}")
        print(f"Galaxy separation: {separation:.2f} arcsec")

        # Visual verification
        plot_centers(dataset.data.native, centers,
                     f"output/plots/{dataset_name}_centers.png")

        # 2. Create mask
        mask = ag.Mask2D.circular(
            shape_native=dataset.shape_native,
            pixel_scales=dataset.pixel_scales,
            radius=mask_radius,
            centre=(0.0, 0.0),
        )
        masked_dataset = dataset.apply_mask(mask=mask)

        # 3. Set up galaxies with custom parameters
        galaxy1 = ag.Galaxy(
            redshift=0.5,
            bulge=ag.lp.Sersic(
                centre=centers[0],
                ell_comps=g1_ell_comps,
                intensity=g1_intensity,
                effective_radius=g1_effective_radius,
                sersic_index=g1_sersic_index,
            ),
        )

        galaxy2 = ag.Galaxy(
            redshift=0.5,
            bulge=ag.lp.Sersic(
                centre=centers[1],
                ell_comps=g2_ell_comps,
                intensity=g2_intensity,
                effective_radius=g2_effective_radius,
                sersic_index=g2_sersic_index,
            ),
        )

        # 4. Fit model
        galaxies = ag.Galaxies(galaxies=[galaxy1, galaxy2])
        fit = ag.FitImaging(dataset=masked_dataset, galaxies=galaxies)
        print(f"Log likelihood: {fit.log_likelihood:.2f}")

        # 5. Calculate intensity ratio
        grid = ag.Grid2D.uniform(
            shape_native=dataset.shape_native,
            pixel_scales=dataset.pixel_scales
        )
```

```
        image1 = galaxy1.bulge.image_2d.from(grid=grid)
        image2 = galaxy2.bulge.image_2d.from(grid=grid)
        total_intensity1 = np.sum(image1)
        total_intensity2 = np.sum(image2)
        intensity_ratio = total_intensity1 / max(total_intensity2, 1e-10)
        print(f"Galaxy 1 total intensity: {total_intensity1:.2e}")
        print(f"Galaxy 2 total intensity: {total_intensity2:.2e}")
        print(f"Intensity ratio (G1/G2): {intensity_ratio:.2f}")

        # 6. Save results
        results = {
            "dataset": dataset_name,
            "separation": separation,
            "intensity_ratio": intensity_ratio,
            "log_likelihood": fit.log_likelihood,
            "galaxy1_center": centers[0],
            "galaxy2_center": centers[1],
            "total_intensity1": total_intensity1,
            "total_intensity2": total_intensity2,
            "g1_ell_comps": g1_ell_comps,
            "g1_intensity": g1_intensity,
            "g1_effective_radius": g1_effective_radius,
            "g1_sersic_index": g1_sersic_index,
            "g2_ell_comps": g2_ell_comps,
            "g2_intensity": g2_intensity,
            "g2_effective_radius": g2_effective_radius,
            "g2_sersic_index": g2_sersic_index,
        }

        # Save results to file
        with open(f"output/results/{dataset_name}_results.txt", "w") as f:
            for key, value in results.items():
                f.write(f"{key}: {value}\n")

        # 7. Save and display fit plot
        try:
            # First create and save the plot
            mat_plot = aplt.MatPlot2D(
                output=aplt.Output(
                    path="output/plots",
                    filename="{dataset_name}_fit",
                    format="png"
                )
            )
            fit_plotter = aplt.FitImagingPlotter(fit=fit, mat_plot_2d=mat_plot)
            fit_plotter.subplot_fit()

            # Now display the fit directly
            display_plot = aplt.MatPlot2D()
            display_plotter = aplt.FitImagingPlotter(fit=fit, mat_plot_2d=display_plot)
            display_plotter.subplot_fit()
            plt.show()

            # Also plot the residuals as a separate figure for better inspection
            plt.figure(figsize=(10, 8))
            plt.imshow(fit.residual_map.native, cmap='coolwarm')
            plt.colorbar(label='Residual')
            plt.title(f"{dataset_name} - Residuals")
            plt.savefig(f"output/plots/{dataset_name}_residuals.png")
            plt.show()

        except Exception as e:
            print(f"Could not create or display fit plot: {e}")

        return results

    except Exception as e:
        print(f"Error analyzing {dataset_name}: {e}")
        return None

# Example usage
if __name__ == "__main__":
    # Choose a dataset to analyze
    if available_datasets:
        # Select first dataset by default
        dataset_to_analyze = available_datasets[0]

        print(f"\nAvailable datasets: {available_datasets}")
        print(f"Analyzing dataset: {dataset_to_analyze}")

        # Run analysis with default parameters
        results = analyze_single_merger(
            dataset_to_analyze,
            # You can modify these parameters as needed for each dataset
            # Galaxy 1 parameters
            g1_ell_comps=(0.1, 0.1),
            g1_intensity=1.0,
            g1_effective_radius=0.5,
            g1_sersic_index=2.5,
            # Galaxy 2 parameters
            g2_ell_comps=(0.1, 0.2),
            g2_intensity=0.3,
            g2_effective_radius=0.4,
            g2_sersic_index=2.0,
            # Peak finding parameters
            min_distance=15,
            sigma=2.0,
            # Mask parameters
            mask_radius=3.0
        )

        print("\nAnalysis complete. Results saved in 'output' directory.")
    else:
        print("No datasets found!")
```